Assignment 3

Chinese Remainder Theorem

Name: Arnav Singhal
MIS: 612203012
Final Year B. Tech. Comp. Div 1
Batch: B3

1. Introduction

The Chinese Remainder Theorem (CRT) is a classical result in number theory that describes how to solve a system of simultaneous linear congruences with pairwise coprime moduli. Given the remainders left by an unknown integer when divided by several pairwise coprime numbers, the theorem guarantees a unique solution modulo the product of those numbers.

2. Mathematical Formulation

Let n1, n2, ..., nk be positive integers that are pairwise coprime, meaning gcd(ni, nj) = 1 for every i not equal to j. Consider the system of congruences:

x = a1 (mod n1), x = a2 (mod n2), ..., x = ak (mod nk)

The Chinese Remainder Theorem states that this system has a solution, and that solution is unique modulo N, where N is the product n1 * n2 * ... * nk. The unique solution in the range [0, N) is given by:

x = (a1*M1*y1 + a2*M2*y2 + ... + ak*Mk*yk) mod N

- Mi = N / ni, the product of all moduli except ni.
- yi is the modular multiplicative inverse of Mi modulo ni, that is, the value satisfying Mi * yi = 1 (mod ni).

3. Algorithm

The accompanying program solves a system of n congruences using the following steps:

- Read the number of congruences n, followed by each remainder ai and modulus ni.
- Verify that every pair of moduli is coprime; if any pair shares a common factor greater than 1, no CRT solution exists and the program reports an error.
- Compute N as the product of all n moduli.
- For each congruence i, compute Mi = N / ni.
- Compute the modular inverse yi of Mi modulo ni using the extended Euclidean algorithm.
- Accumulate x = x + ai * Mi * yi for every congruence, then reduce the final x modulo N.
- Verify the result by checking that x mod ni equals ai for every congruence.

The extended Euclidean algorithm is used to compute both the greatest common divisor of two numbers and the coefficients of Bezout's identity, from which the modular inverse is directly obtained.

4. Worked Example

Consider the following system of three congruences:

x = 2 (mod 3), x = 3 (mod 5), x = 2 (mod 7)

Since 3, 5 and 7 are pairwise coprime, N = 3 * 5 * 7 = 105. The table below shows the intermediate values computed for each congruence:

i	ai	ni	Mi = N/ni	yi = Mi^-1 mod ni	ai*Mi*yi
1	2	3	35	2	140
2	3	5	21	1	63
3	2	7	15	1	30

Summing these values gives 140 + 63 + 30 = 233. Reducing modulo N = 105 gives the final solution:

x = 233 mod 105 = 23

Verification: 23 mod 3 = 2, 23 mod 5 = 3, and 23 mod 7 = 2, which matches every original congruence, confirming that x = 23 is the unique solution modulo 105.

5. Pairwise Coprimality Requirement

The Chinese Remainder Theorem only guarantees a solution when the moduli are pairwise coprime. If two moduli share a common factor greater than 1, the corresponding pair of congruences may be inconsistent, or if consistent, the combined system no longer has a unique solution modulo the product of the moduli.

Before computing a solution, the program checks gcd(ni, nj) for every pair of moduli using the extended Euclidean algorithm. If any pair is not coprime, the program raises an error instead of returning an incorrect or misleading result.

6. Program Implementation

6.1 Core Functions

The program (crt.py) is built around three core functions: extended_gcd(), which returns the greatest common divisor of two integers together with Bezout coefficients; mod_inverse(), which uses extended_gcd() to compute the modular inverse of a number, returning None if no inverse exists; and crt(), which implements the full algorithm described in Section 3, including the pairwise coprimality check.

6.2 Input and Output Design

The program is interactive: it first asks the user for the number of congruences n, then reads each remainder and modulus pair in turn through read_congruences(). After computing the solution, it prints the original system of congruences, the final solution x modulo N, and a verification step that re-checks x against every modulus, so correctness can be confirmed by inspection of the output.

6.3 Code and Output

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return g, x, y



def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        return None
    return x % m



def crt(remainders, moduli):
    n = len(moduli)
    if n != len(remainders):
        raise ValueError("remainders and moduli must have the same length")

    for i in range(n):
        for j in range(i + 1, n):
            g, _, _ = extended_gcd(moduli[i], moduli[j])
            if g != 1:
                raise ValueError(
                    f"moduli {moduli[i]} and {moduli[j]} are not pairwise coprime"
                )

    M = 1
    for m in moduli:
        M *= m

    x = 0
    for r, m in zip(remainders, moduli):
```

```
        Mi = M // m
        Mi_inv = mod_inverse(Mi % m, m)
        x += r * Mi * Mi_inv

    return x % M, M


def read_congruences():
    n = int(input("Enter the number of congruences: "))
    remainders = []
    moduli = []
    for i in range(n):
        r = int(input(f"Enter remainder a{i + 1} (x = a{i + 1} mod n{i + 1}): "))
        m = int(input(f"Enter modulus n{i + 1}: "))
        remainders.append(r)
        moduli.append(m)
    return remainders, moduli


def main():
    remainders, moduli = read_congruences()
    try:
        x, M = crt(remainders, moduli)
    except ValueError as e:
        print("Error:", e)
        return

    print()
    print("System of congruences:")
    for r, m in zip(remainders, moduli):
        print(f"  x = {r} (mod {m})")

    print()
    print(f"Solution: x = {x} (mod {M})")

    print()
    print("Verification:")
    for r, m in zip(remainders, moduli):
        print(f"  {x} mod {m} = {x % m}  (expected {r})")
```

```
if __name__ == "__main__":
    main()
```

Output:

```
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment3$ python3 crt.py
Enter the number of congruences: 3
Enter remainder a1 (x = a1 mod n1): 2
Enter modulus n1: 3
Enter remainder a2 (x = a2 mod n2): 3
Enter modulus n2: 5
Enter remainder a3 (x = a3 mod n3): 2
Enter modulus n3: 7

System of congruences:
  x = 2 (mod 3)
  x = 3 (mod 5)
  x = 2 (mod 7)

Solution: x = 23 (mod 105)

Verification:
  23 mod 3 = 2  (expected 2)
  23 mod 5 = 3  (expected 3)
  23 mod 7 = 2  (expected 2)
arnav@arnav-HP-Laptop-15s-fq5xxx:~/Desktop/COEP/CNS/Assignment3$
```

6.4 Demonstration Result

When run with the sample system x = 2 (mod 3), x = 3 (mod 5), x = 2 (mod 7), the program correctly reports the solution x = 23 (mod 105). The verification step confirms that 23 leaves remainder 2 when divided by 3, remainder 3 when divided by 5, and remainder 2 when divided by 7, exactly matching the original congruences and confirming the correctness of the implementation.

7. Conclusion

The Chinese Remainder Theorem provides an efficient way to reconstruct a unique integer from its remainders modulo a set of pairwise coprime numbers. The accompanying program generalizes the classical two-congruence case to an arbitrary number n of congruences, validating pairwise coprimality before combining the congruences using modular inverses computed via the extended Euclidean algorithm. The worked example and program output confirm that the implementation produces correct and verifiable results.